

# Analysis: Endless Knight

The small dataset can be solved by simple, two-dimensional dynamic programming. In fact, this is obviously the easiest dataset most of the contestants found in this round, with 884 correct submissions. Quite on the contrary, the large dataset turns out to be the most difficulty for this round, solved by only 32 contestants.

Below we outline three tricks one may use in this problem.

## Section A. A nice simplification

Most people are familiar with lattice walks from  $(0, 0)$  to  $(m, n)$ , where each step one can either increase row by 1 or increase column by 1. The total number of such walks is the binomial number  $\binom{m+n}{n}$ . One simple reason is that the set of paths is bijective to the ways you choose  $m$  steps for vertical moves among the  $(m+n)$  steps.

If there are no rocks, one can see that the situation in this problem is quite similar. In fact, they are essentially the same. After a nice transformation of the input, we can forget about the knight and focus on the normal lattice walks.

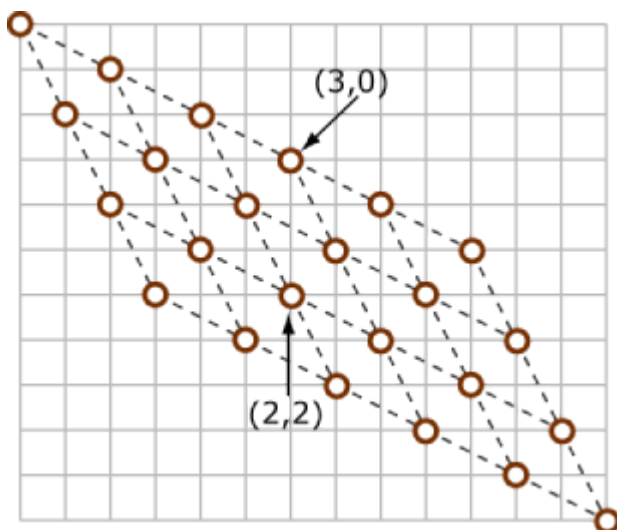
In our problem, the two kinds of moves the knight can make correspond to the vectors  $u = (1, 2)$  and  $v = (2, 1)$ . The set of reachable positions forms a lattice with  $u, v$  as the basis. To find the coordinate of  $(r, c)$  in the new system is a matter of linear transformation between two bases. In our problem, it is as simple as

$$r'(2, 1) + c'(1, 2) = (r, c) - (1, 1).$$

Solving, we get  $r+c = 2 \pmod 3$ , and

$$r' = r - 1 - (r+c-2)/3, \text{ and } c' = c - 1 - (r+c-2)/3.$$

We illustrate using the following picture.



When reading the input, we can transform the points to the new system, and throw away any points that are not reachable. Assume the destination is reachable, otherwise we can simply output 0. We can also disregard any rock with row number or column number that exceeds the destination. In short, we are now in a rectangular lattice in the new coordinate system.

## Section B. A key idea

Notice that there is an important restriction in our problem: there are at most 10 rocks. The key idea to our solution -- although other solutions are possible as well -- is the inclusion-exclusion principle.

Let  $S$  be any subset of rocks (including the empty set). Define  $f(S)$  to be the number of ways we can walk from the origin to destination and hitting every rock in  $S$ , and possibly hitting some other rocks.

$$\text{Number of paths that do not hit any rock} = \sum_S f(S) (-1)^{|S|}.$$

*Also note:* we refer to the Round 1C problem Ugly Numbers. There we needed to consider the multiples of 2, 3, 5, and 7. Some solutions can be viewed as an application of the inclusion-exclusion principle, although this is not necessary for solving that problem.

## Section C. $N$ choose $K$ mod 10007?

Now we need to calculate  $f(S)$  for any  $S$ . Let's sort rocks in  $S$  from left to right, and for rocks on the same column, pick the higher one first. It should be clear that if some later rock is higher than some earlier ones, then  $f(S) = 0$  -- there is no way to hit all the rocks in  $S$ . Otherwise, the sorted set  $S$  forms a chain from the top-left corner to the lower-right corner

$$(0, 0) =: (r_0, c_0) \rightarrow (r_1, c_1) \rightarrow \dots \rightarrow (r_{k+1}, c_{k+1}) := \text{destination}$$

We can view any path hitting all the  $k$  rocks as  $(k+1)$  stages.  $f(S)$  is the product of the number of ways we can do each stage. (This is another important counting principle. It is so important and obvious that usually people don't call it by name. But it does have a name -- the multiplication rule.)

Now, we are back to the classical problem in the beginning of this analysis. Let  $m = (r_i - r_{i-1})$  and  $n = (c_i - c_{i-1})$ , the number of ways we can do the  $i$ -th stage is exactly  $(m+n)$  choose  $n$ .

So, is this the end of the story? Not yet. Many contestants failed this problem because it is tricky to compute  $A$  choose  $B$  mod 10007 quickly and correctly. In this problem, both  $A$  and  $B$  can be on the order of  $10^8$ . To compute  $A$  choose  $B$  mod  $P$  for a prime number  $P$ , one needs some tricks. There are many clever ways you can find, like pre-computing  $N!$  for all  $N$ , pre-computing the inverse of each number mod  $P$ , utilizing the periodicity of the numbers in factorials. What we want to introduce below is a nice theorem that is not as well known as it should be. It removes all the worries about the multiples of  $P$ . It is not a difficult theorem, but looks very cute and especially useful for this problem.

**Lucas' Theorem:** Suppose  $(n_i n_{i-1} \dots n_0)$  and  $(k_i k_{i-1} \dots k_0)$  are the representation of  $n$  and  $k$  in base  $P$ , where  $P$  is a prime number. Then  $(n \text{ choose } k)$  is the product of  $(n_i \text{ choose } k_i)$  in  $Z_P$ .

The following is the choose function from **Reid's** beautiful Haskell code.

```
choose :: Int -> Int -> Int10007
choose n k | k > n = 0
choose n k | n < 10007 =
    product [ (fromIntegral i) :: Int10007 | i <- [n-k+1..n] ]
    / product [ (fromIntegral i) :: Int10007 | i <- [1..k] ]
choose n k = choose qn qk * choose rn rk
    where (qn, rn) = n $$$ \mathbf{divMod}$$$ 10007
          (qk, rk) = k $$$ \mathbf{divMod}$$$ 10007
```

## More information

[The inclusion-exclusion principle](#) - [Lucas' Theorem](#) - [Staircase walk](#)

For the study of lattice points, we refer to any standard text in Discrete Geometry.

# Analysis: How Big Are the Pockets?

## (i). Computing the area of the polygon

As Polygonovich walks, the interior of the polygon either always stays on his right side, or always on his left side. This is by no means a trivial fact. The polygon can be so complicated that given a portion of the walk in the middle, one has no way to decide whether the left side is the interior, or the right side is. Let us accept this fact and assume the former case -- his right hand always touches the interior of the polygon.

Fix a vertical bar of unit width, observe the interaction between  $P$ , Polygonovich's walk, and  $B$ , the vertical bar.

- $P$  crosses  $B$  an even number of times. Because the walk is closed, and any time Polygonovich crosses from the left to the right, the next time he must cross in the other direction. From top down, we label them as the 1st crossing, the 2nd one, and so on.
- Furthermore, look down at  $B$  from high above. In the beginning, it is outside the polygon. Every time it encounters an edge of  $P$ , it changes from being outside the polygon to being inside and vice versa. So, a unit square on  $B$  is inside the polygon if and only if it is between some  $(2k-1)$ -st crossing and  $(2k)$ -th crossing.
- Note that if the polygon is always on the right-hand side, then we know that the  $(2k-1)$ -st crossing is always from left to right, and the  $(2k)$ -th one is from right to left.

Given any unit square  $U$ , we can decide whether it is in the polygon by counting the number of left-to-right crossings vertically above  $U$ , minus the number of right-to-left crossings above  $U$ . We can have a little counter inside  $U$ , and each time there is a left-to-right crossing above  $U$ , we increase the counter by 1; and for the right-to-left crossings, decrease the counter by 1.

After these mental exercises, we make the final jump back to the interaction between  $P$  and  $B$ . We may imagine a bounding box outside the polygon so that there are finitely many counters, and we now consider the area, it is nothing other than the summation of all the counters. To compute the area, we do the following.

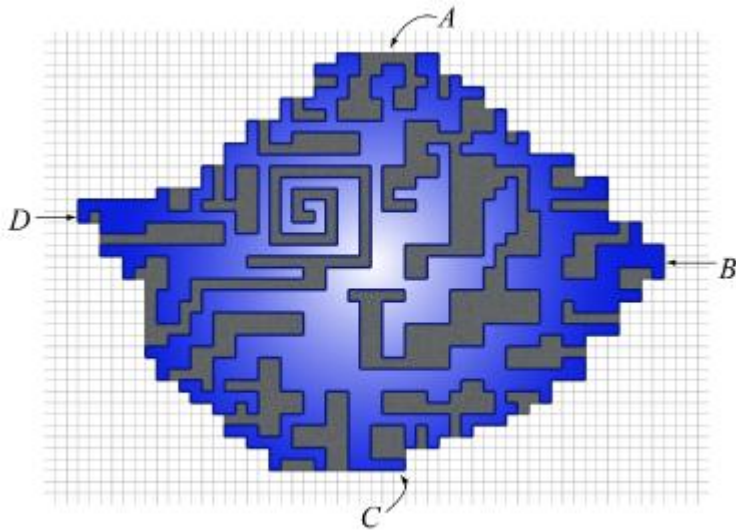
Set  $A = 0$  in the beginning. Walk along  $P$ , for each horizontal edge from left to right at height  $h$ , we increment  $A$  by  $h$ . This is the effect of this edge on all the counters for the unit squares below. And for each edge from right to left at height  $h$ , we decrement  $A$  by  $h$ .

All the time we assume the interior is always on the right hand side. In the opposite case, the highest edge on each bar will be a right-to-left cross, and all of the reasoning above is similar with only a difference in signs. So if we ever find that  $A$  is negative at the end, we can negate it and get the right area.

We note that this is just a special case of the simple algorithm for computing polygon areas in general. However, isn't the pictures nice, in the special form of integer grids and axis-parallel edges?

## (ii). Computing the area of the polygon plus the pockets

The polygon plus the pockets gives staircases in four (NE, NW, SE, SW) directions. A formal proof would be tedious. A picture with a good example should suffice.



As in the picture, let  $A$  be any one of the topmost edges,  $C$  be the bottommost edge,  $B$  the rightmost, and  $D$  the left-most one. We have 4 staircases: one from  $A$  to  $B$ , one from  $B$  to  $C$ , one from  $C$  to  $D$ , and one from  $D$  to  $A$ . Theoretically speaking, these are formed by the maximal points with respect to four directions, and each of these staircases can be computed in  $m \log m$  time, where  $m$  is the number of points on the polygon. In this problem, we use the guarantee that there are at most 6000 vertical strips. For each vertical strip  $x$ , define

$t(x) :=$  the topmost polygon edge to cross the strip.

$b(x) :=$  the bottommost polygon edge to cross the strip.

$H(x) :=$  the topmost polygon edge or pocket on that strip.

$L(x) :=$  the bottommost polygon edge or pocket on that strip.

For  $x$  between  $A$  and  $B$ ,  $H(x)$  is the maximum  $t(x')$  for all  $x'$  between  $x$  and  $B$ . We may compute  $H(x)$  as follows

```
H(x) = t(x) for the last strip
for x = (the second last strip) down to A
    H(x) = max(H(x+1), t(x))
```

The values on the other staircases can be computed in the same manner.

On strip  $x$ , the polygon plus pockets are the unit squares between  $L(x)$  and  $H(x)$ . So we can sum  $H(x) - L(x)$  over all the 6000 possible strips and get the area of the polygon with pockets.

### (iii). Solve our problem

(ii) minus (i).

# Analysis: No Cheating

Programming contests have certainly changed over the years. Nowadays, dynamic programming is a trivial matter to many, and bipartite matching is no longer a secret idea. More than half of the contestants solved the small datasets of this problem, and 77 passed the large tests.

### Small dataset (Test set 1 - Visible)

The small tests can be solved using dynamic programming. The main idea is to do this row by row.

The naive dynamic programming has states  $(R1, r)$ , where  $r$  is the row number and  $R1$  is the possible configurations of the current row. In order to compute the value of  $(R1, r)$ , (the maximum number of students who can be put in the first  $r$  rows, with the  $r$ -th row identical to  $R1$ ), one can loop through all of the possible values of  $R2$  and use the value of  $(R2, r-1)$ .

A more sophisticated approach also goes row by row, but in each row, we do it by squares. The states are  $(S, r, c)$ , where  $(r, c)$  is the current position, and  $S$  is the bitmap for up to  $n$  positions before  $(r, c)$ . These are all the squares to the left on the same row, and all the squares from the  $(c-1)$ -st to the last on the last row. In this way, each value of  $(S, r, c)$  can be computed in constant time.

One may estimate the number of states to be  $O(M N 2^N)$ . But actually the number of states is much smaller. It is not hard to see, with the restriction that no two horizontally adjacent seats can be both occupied, that the number of states  $S$  is  $O(F_N)$ , the  $N$ -th Fibonacci number. This means that the dynamic programming solution can even handle a classroom of size about 35 by 35.

### Large dataset (Test set 2 - Hidden)

Well, but universities do have very large classrooms. The large dataset has tests with 80 rows and 80 columns.

The key idea is to see through the puzzle and notice that this is a standard graph problem. Let each square be a node. What the problem requires is to pick a set of nodes, such that some pairs of nodes cannot be picked together. Let us draw an edge between each such pair of nodes. This is the well-known problem of finding a maximum independent set of the graph. While the general independent set problem is NP-hard, we do know that the problem is tractable on bipartite graphs. A-ha! The graph is indeed bipartite, because all of the edges are between a square in odd-numbered columns and squares in even-numbered columns.

We can easily find the [maximum matching](#) on bipartite graphs. Let's see how a maximum matching can help us solve the problem. Let's assume that the size of the matching is  $X$ , and the number of nodes in the graph is  $N$ . Notice that the the maximum independent set cannot contain more than  $N - X$  nodes, because that would mean you will have two nodes that share an edge of the maximum matching in this set.

In general, we have the following relations between the important graph parameters. For a graph  $G = (V, E)$ , let  $\alpha(G)$  be the size of the maximum independent set,  $m(G)$  be the size of a maximum matching,  $\mu(G)$  be the size of a minimum vertex cover,

- The complement of an independent set is a vertex cover, and vice versa. So  $\alpha(G) + \mu(G) = |V|$ .
- To cover all the edges, we need one vertex from any edge of a matching. So  $\mu(G) \geq m(G)$ .
- If  $G$  is bipartite, then  $\mu(G) = m(G)$ . Equivalently,  $\alpha(G) + m(G) = |V|$ .

The last fact can be proved directly, or as a consequence of Hall's theorem, or as an application of the powerful max-flow-min-cut theorem. Below we provide a constructive proof that suits to our problem.

## Proof

We will prove that  $N - X$  is smaller than or equal to the size of the maximum independent set. Let's have a maximum matching  $M$  and a set  $S$  which contains all the nodes that are not in  $M$ . Now let's add to  $S$  all the nodes from  $M$  which are on the left side of the bipartite graph.  $S$  is of size  $N - X$ . If  $S$  is not an independent set, then it has a node  $u$  from  $M$  and another one  $v$  that doesn't belong to  $M$ , such that  $(u, v)$  is an edge. To fix this problem we remove  $u$  from  $S$  and add node  $u'$  to  $S$  where  $(u, u')$  is an edge from the matching. If this operation caused a problem with another node in the matching, then we can remove that node and add its match, and so on. This repeated procedure is guaranteed to finish since we only delete nodes from the left side of the graph and add nodes from the right side. There will be no problem caused by the nodes on the right side of the graph and nodes outside the matching because that would mean that we can find an alternating path, and the maximum matching is not the largest possible.

We have proved that in any bipartite graph, the size of the maximum independent set equals the size of the graph minus the size of the maximum matching.

## Code

We believe that many contestants have the standard bipartite matching algorithm prepared. You may download many nice samples from the scoreboard. Below we show a sample solution which keeps the bipartite graph only in mind, and actually runs the matching algorithm on the board itself. From the judges:

```
int C, N, M;
string bd[100];

int nbx[100][100], nby[100][100], v[100][100];
int T=0;

bool dfs(int a, int b) {
    if (a<0) return true;
    if(v[a][b]==T) return false;
    v[a][b]=T;
    for (int i=a-1;i<=a+1;i++)
        for (int j=b-1;j<=b+1;j+=2)
            if (i>=0 && i<M && j>=0 && j<N && bd[i][j]=='.') {
                if (dfs(nbx[i][j], nby[i][j])) {
                    nbx[i][j]=a; nby[i][j]=b;
                    nbx[a][b]=i; nby[a][b]=j;
                    return true;
                }
            }
    return false;
}

int play() {
```

```

memset(nbx,-1,sizeof(nbx));
memset(nby,-1,sizeof(nby));
memset(v, -1, sizeof(v)); T=-1;
int rst=0;
for(int i=0;i<M;i++) for(int j=0;j<N;j++) {
    if (bd[i][j]=='.') {
        rst++;
        if (j%2) {
            T++;
            if (dfs(i,j)) rst--;
        }
    }
}
return rst;
}

int main() {
    cin>>C;
    for (int i=1; i<=C; i++) {
        cin>>M>>N;
        for (int r=0;r<M;r++) cin>>bd[r];
        cout<<"Case #"<<i<<": "<<play()<<endl;
    }
    return 0;
}

```

## More Information

[Maximum independent set](#) - [Bipartite Matching](#)



# Analysis: Portal

The challenge in this problem was mainly coding a bug-free solution. It was pretty obvious that solving it involved using a shortest path algorithm. And as you can see, even some people in the top 20 skipped this problem to go for other ones, which were more difficult to figure out, but easier to code.

Let's look at **bmerry**'s solution, because it is very readable, and he was the winner of this round. Then I will continue with some other, more efficient solutions.

A state of the game corresponds to the position of the player in the maze and the positions of the two portals, if they exist. This solution considers the map of the maze indexed from 1 so the values (0, 0) for the coordinates of the portals mean that the portals do not exist.

At each step, the player can create a new portal, move one step North, South, East or West or, if he is currently near a portal, and another portal exists, travel from one portal to the other one. The first type of move can be made instantaneously while the second and third types take one turn.

One optimization step is to find, for each cell, the positions of the portals that can be created from that cell, since you don't want to take  $O(R + C)$  every time you need to find the possible moves from a state.

If each move took exactly one turn, then the classic breadth-first search algorithm could provide us with the answer, but in this graph with two types of weights for the edges it seems like we need Dijkstra's shortest path algorithm. In fact, we can still use an algorithm very similar to breadth first search. The tweak is that instead of adding a new state of the same cost with the current state at the end of the state queue, we add it at the beginning. This way the states will be expanded in the order of their costs, which is exactly what Dijkstra's algorithm does. The complexity of this algorithm is  $O((R*C)^3)$  instead of  $O((R*C)^3 \log(R*C))$ , which is the cost of Dijkstra's shortest path algorithm.

## Code

Here is **bmerry**'s code, modified slightly and with some additional comments.

```
struct state {
    int r;
    int c;
    // portal rows
    int pr[2];
    // portal columns
    int pc[2];
};

#define ADDR(state) state.r][state.c] \
                    [state.pr[0]][state.pc[0]] \
                    [state.pr[1]][state.pc[1]]

static unsigned char prio[16][16][16][16][16][16];

static const int dr[4] = {-1, 0, 1, 0};
static const int dc[4] = {0, -1, 0, 1};
```



```

        // is instantaneous.
        q.push_front(nxt);
    }
}

for (int d = 0; d < 4; d++) {
    state nxt = cur;
    nxt.r += dr[d];
    nxt.c += dc[d];
    if (grid[nxt.r][nxt.c] != '#') {
        if (prio[ADDR(nxt)] > pri + 1) {
            prio[ADDR(nxt)] = pri + 1;
            q.push_back(nxt);
        }
    }
}

if (cur.pr[0] > 0 && cur.pr[1] > 0)
    for (int p = 0; p < 2; p++)
        if (cur.pr[p] == cur.r &&
            cur.pc[p] == cur.c) {
            state nxt = cur;
            nxt.r = cur.pr[1 - p];
            nxt.c = cur.pc[1 - p];
            if (prio[ADDR(nxt)] > pri + 1) {
                prio[ADDR(nxt)] = pri + 1;
                q.push_back(nxt);
            }
        }
}

printf("Case #%d: ", cas + 1);
if (ans == -1)
    printf("THE CAKE IS A LIE\n");
else
    printf("%d\n", ans);
}
return 0;
}

```

The restrictions on the inputs in the problem are small enough so that this solution passes all the tests. There are other, more efficient solutions which we thought of.

## Other solutions

It does not make sense to create the starting portal before actually being able to jump through it. This way we can improve the previous solution and get the complexity down to  $O((R \cdot C)^2)$ .

As we have just seen, premature portal creation is the root of complexity, so let's think now of the destination portal. Once we have created a destination portal, we need to move as quickly as possible to the nearest wall, create a starting portal, walk through it and arrive at the destination. So what we can do is just keep  $R \cdot C$  states and move from one to another by either going North, West, South or East or do a teleport move which takes a few turns. We can compute in  $O(R \cdot C)$  how much time each teleport move takes by doing a breadth first search starting from all the cells in the maze where a portal can be created. Now we can use Dijkstra's algorithm to find the shortest path. The final algorithm can have either  $O(R \cdot C \log (R \cdot C))$

complexity, or if we use the fact that the teleport edges have the cost at most  $R * C$ , by using an array of lists in Dijkstra's algorithm instead of a priority queue, we can get the complexity of the solution down to  $O(R * C)$ .

## More Information

[Dijkstra's algorithm](#) - [Breadth First Search](#)